

CS 122A Custom Laboratory Project Report

FPGA Tetris

Marlon Lopez, Chris Antony

06/10/2026

Introduction

This project is a playable Tetris game that runs on an iCESugar Pro FPGA and is driven by a handheld controller built around a Raspberry Pi Pico. The player uses a joystick and two push buttons connected to the Pico to move, rotate, drop, and swap the falling pieces. The Pico reads these inputs and sends them to the FPGA, which runs the entire game and draws the playfield to an HDMI monitor in real time.

The game implemented standard tetris gameplay features successfully: Piece movement, rotation, fast drop, lock delay, row clearing, piece preview, hold/swap, random piece selection, and a game over state.

Elements of Complexity

HDMI - Successfully Implemented

Our project uses HDMI to display the game onto a display. We consider this an element of complexity as it had its own driver we had to interface with. We produced a 640 x 480 video signal to view gameplay on.

SPI - Successfully Implemented

Our Pico communicates with our FPGA using SPI. This ensures that the FPGA only has to deal with running the game, while the Pico can focus on receiving and sending the inputs.

Tetris Engine - Successfully implemented

The game logic was written in Verilog as a synchronous finite state machine.

Random piece generation - Successfully implemented

A linear feedback shift register was used during active gameplay in order to have a random game piece appear next.

LCD Scoreboard - Not implemented

User Guide

The user plays Tetris on the HDMI monitor using the joystick and buttons on the Pico-based controller. After both boards are powered on, the FPGA starts the game automatically and the first piece begins to fall.

The player sees a 640 x 480 screen where a 10 column by 20 row playfield is displayed. The playfield has a dark green background, with green grid lines. The active falling piece is drawn in red and a dropped block is drawn in gray. A preview block is shown on the top right side of the playfield, that shows the next block to spawn. A stored box appears on the top left showing the piece currently set aside for swapping.

Moving the joystick left and right causes the piece to move accordingly with a delay of a second. The playing area has walls to stop the pieces from moving out of bounds. Pressing the joystick button will store the piece, if there is none stored already. If there is, it swaps out the current piece for the stored one, and puts it at the top. This can only be done once per piece to prevent stalling.

Pressing the left button will cause the piece to drop faster, otherwise known as “fast drop”. Pressing the right button will cause the piece to rotate clockwise.

The game will continue until pieces reach the top row. When this happens, the game will clear and start over from the beginning.

Hardware Components Used

- iCESugar Pro
- iCESugar Pro Expansion Board
- Pi Pico
- Joystick
- HDMI Monitor
- Buttons
- Breadboard and jumper wires

Software Libraries

N/A

Protocols Used

SPI

Used to send the player's command from the Pico to the FPGA.

The Pico acts as the SPI master and the FPGA as the receiver; only the clock, chip-select, and MOSI lines are used because data flows in one direction. The Pico transmits one 8-bit command byte per update (roughly every 20 ms) at 1 MHz, framed by the chip-select line. This byte tells the FPGA which action the player requested (move, rotate, drop, or swap).

TMDS / HDMI (DVI).

Used to drive the display. The FPGA encodes each color channel of the 24-bit pixel data using TMDS, then serializes the three color channels plus a clock onto differential pairs through differential output buffers, producing a standard 640x480 HDMI/DVI signal that the monitor can show directly

Requirements

List the 4 requirements mentioned in the proposal page and explain what you did for each one.

Icesugar/Icesugar-pro FPGA

The gameplay logic runs on the iCESugar. The FPGA receives player inputs from the pico over SPI communication, and generates the 640 x 480 video, using TMDS encoding to drive the HDMI monitor.

RGB LCD from the required lab components (HDMI display in our case)

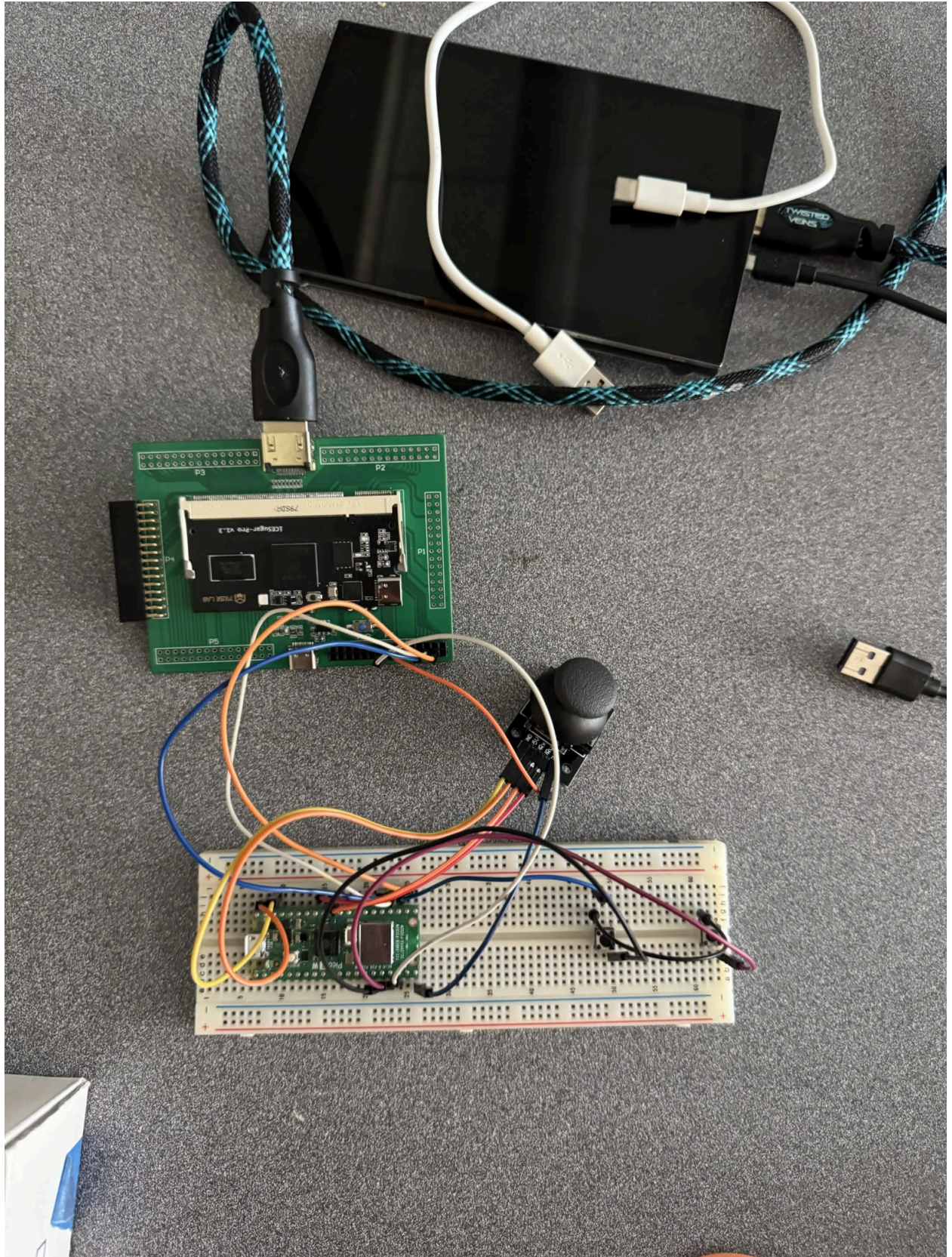
The HDMI monitor displays the game to the player. We drive the monitor at 640 x 480 resolution at 60 hz by the FPGA.

Raspberry Pi Pico

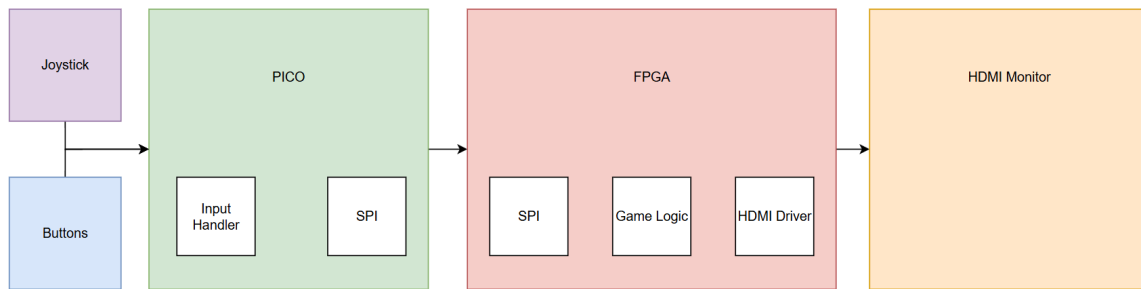
The pico acts as our input handler. It polls the joystick (ADC) and buttons (GPIO) for inputs, converts them to data, and sends it to the FPGA over SPI as the SPI master.

Event driven execution(timing call backs are a type of event driven execution)

We used event driven execution for game updates. On the pico, we sample our controller, encode a command byte, and send it over spi every 20 ms. The iCESugar updates the game state machine and is driven by timer events from the 25 MHz clock, where the piece falling down speed happens every 1s, move happens every 150 ms, fast drop every 50 ms, and lock at 0.5 s. HDMI is updated using the 25 MHz clock.



Design Diagram



AI Usage

Used AI to debug minor issues with Verilog.

Acknowledgements

HDMI Code: https://github.com/wuxx/icesugar-pro/tree/master/src/hdmi_test_pattern